server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

1. Install the pipeline; the default ships nothing

As root, run server2/deploy/logging/install.sh (with --collector-tarball for a local copy of the pinned archive, otherwise it downloads it). It checks the archive hash, installs the units and enables them. With no SERVER2_LOG_SINKS setting, start both units and check the collector.

```
$ bash server2/deploy/logging/install.sh --collector-tarball
    reports/lp-2977/vendor/vector-0.61.0-aarch64-unknown-linux-musl.tar.gz
install.sh: installed vector 0.61.0 (aarch64-unknown-linux-musl 17e017e 2026-10-29
    16:03:40.929269968)

$ systemctl start server2-logs.service server2.service; systemctl is-active server2.service
active

$ systemctl status server2-logs.service --no-pager | grep -E 'server2-logs.service
    -|Active:|Condition'
○ server2-logs.service - server2 log collector
     Active: inactive (dead) (Result: exec-condition) since Sat 2026-11-14 15:00:07 KST; 77ms ago
  Condition: start condition unmet at Sat 2026-11-14 15:00:07 KST; 135ms ago
    Process: 15185 ExecCondition=/usr/bin/python3 -I /opt/server2/logging/render.py --env-file
    /opt/server2/config/.env --state /var/lib/server2-logs --output /run/server2-logs/vector.json
    --version-file /opt/server2/logging/collector.json (code=exited, status=1/FAILURE)
Nov 14 15:00:07 lima-lp2977 systemd[1]: Starting server2-logs.service - server2 log collector...
Nov 14 15:00:07 lima-lp2977 systemd[1]: Condition check resulted in server2-logs.service - server2
    log collector being skipped.

$ journalctl --namespace=server2 -u server2-logs.service -o cat
server2 log collector: only the local journal is selected; nothing is shipped
```

install.sh reports the collector version. server2.service is active; server2-logs.service is skipped by its start condition and says why in the server2 journal.

Expected result: The server runs and writes to the local journal namespace server2. The collector is skipped (start condition unmet), so nothing leaves the host.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

2. A mistyped setting fails only the collector

Set SERVER2_LOG_SINKS in /opt/server2/config/.env (here with a typo, clodwatch) and start the units. Read the collector's message in the server2 journal namespace, fix the value, and start the collector again.

```
$ grep '^SERVER2_LOG' /opt/server2/config/.env
SERVER2_LOG_SINKS=s3,clodwatch
SERVER2_LOG_S3_PREFIX=logs/server2

$ systemctl start server2-logs.service server2.service; systemctl is-active server2-logs.service
    server2.service
Job for server2-logs.service failed because the control process exited with error code.
See "systemctl status server2-logs.service" and "journalctl -xeu server2-logs.service" for details.
failed
active

$ journalctl --namespace=server2 -u server2-logs.service -o cat
server2 log collector: invalid setting SERVER2_LOG_SINKS: unknown destination; allowed: local, s3,
    cloudwatch; the collector is not started

$ curl -s -o /dev/null -w 'health %{http_code}\n' http://127.0.0.1:18090/health
health 200
```

The collector is failed and names SERVER2_LOG_SINKS and the allowed values, without repeating the mistyped value. The server is active and /health answers 200.

Expected result: server2-logs.service is failed without restarting in a loop; server2.service keeps serving.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

3. Ship to S3 and CloudWatch Logs with different levels

Set the destinations, the per-destination levels, the S3 prefix and an existing CloudWatch log group in .env, then start (or restart) the collector. On EC2 the credentials come from the instance role.

```
$ grep '^SERVER2_LOG' /opt/server2/config/.env
SERVER2_LOG_SINKS=local,s3,cloudwatch
SERVER2_LOG_S3_LEVEL=info
SERVER2_LOG_CLOUDWATCH_LEVEL=warn
SERVER2_LOG_S3_PREFIX=logs/server2
SERVER2_LOG_CLOUDWATCH_GROUP=/server2/production

$ systemctl start server2-logs.service server2.service; sleep 2; systemctl is-active
    server2-logs.service server2.service
active
active

$ journalctl --namespace=server2 -u server2-logs.service -o cat | grep collector.started | jq -c .
{"boot_id":"f285e857880a4a03b545bbefa2e2a1ef","collector_schema":1,"collector_version":"vector 0.61.
    0","event":"collector.started","instance_id":"b09bf0476120469f9b0cd4349a3edcd1","level":"info","
    sinks":["local","s3","cloudwatch"],"timestamp":"2026-11-14T06:00:13.140800Z"}
```

Both units are active. The collector's first record names its version, the active destinations, the host and the boot. This capture predates LP-3: the record now also carries instance_life_cycle, and the host is the EC2 instance ID (the machine ID, as here, only without instance metadata); see step 8.

Expected result: The collector start record appears in the journal and is shipped to every remote destination.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

4. Where records arrive

Serve requests, then stop the server and the collector (systemctl stop server2.service server2-logs.service stops the server first). The collector's stop waits until every destination has confirmed the records (up to about one S3 object window, 50 s since LP-4). Look at what arrived: S3 keys under the prefix and CloudWatch streams in the group.

```
# What the stand-in received in place of S3 (keys) and CloudWatch Logs (streams)
$ cd /var/tmp/manual-standin/s3 && find . -type f | sed 's#^./##'
logs/server2/collector/dt=2026-11-
    14/hour=06/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef-20261114T060021Z-
    d559a150-0030-4596-b9dd-bd087ef75553.ndjson.gz
logs/server2/server/dt=2026-11-
    14/hour=06/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef-20261114T060021Z-
    1bfd615c-2b1c-4eb1-8234-313ae4e0b05b.ndjson.gz

$ cd /var/tmp/manual-standin/cloudwatch/server2/production && find . -type f | sed 's#^./##;
    s#.jsonl$##'
collector/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef
server/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef
```

S3 keys and CloudWatch streams carry the source (server or collector), the host ID and the boot ID; S3 keys also carry the UTC date and hour and a unique suffix.

Expected result: Server and collector records are kept apart, and records from different hosts or boots never share a stream or key.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

5. Read each destination

Read the local journal with journalctl --namespace=server2, an S3 object with gzip -dc (after aws s3 cp), and the CloudWatch stream (aws logs tail on EC2). Filter with jq.

```
$ journalctl --namespace=server2 -u server2.service -o cat | grep -v '^{'
server2 public=127.0.0.1:18090 metrics=127.0.0.1:18091

$ journalctl --namespace=server2 -u server2.service -o cat | grep '^{' | jq -c
    'select(.level=="warn") | {timestamp,level,component,stage,status} | with_entries(select(.value
    != null))'
{"timestamp":"2026-11-
    14T06:00:16.245834599Z","level":"warn","component":"gateway","stage":"provider_http"}
{"timestamp":"2026-11-
    14T06:00:16.248366929Z","level":"warn","component":"http","stage":"request","status":501}

# S3 object (gzip NDJSON), read as with: aws s3 cp s3://BUCKET/KEY - | gzip -dc
$ gzip -dc /var/tmp/manual-standin/s3/logs/server2/server/dt=2026-11-
    14/hour=06/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef-20261114T060021Z-
    1bfd615c-2b1c-4eb1-8234-313ae4e0b05b.ndjson.gz | jq -c '{level,event,component,stage,status} |
    with_entries(select(.value != null))' | head -n 8
{"level":"info","event":"operation.action","component":"process","stage":"startup"}
{"level":"info","event":"operation.action","component":"process","stage":"bind"}
{"level":"info","event":"operation.started","component":"http","stage":"request"}
{"level":"info","event":"operation.started","component":"gateway","stage":"request"}
{"level":"warn","event":"operation.finished","component":"gateway","stage":"provider_http"}
{"level":"info","event":"operation.finished","component":"gateway","stage":"request"}
{"level":"warn","event":"operation.finished","component":"http","stage":"request","status":501}
{"level":"info","event":"operation.action","component":"process","stage":"shutdown"}

# CloudWatch Logs stream server/<instance>/<boot>, read as with: aws logs tail GROUP --log-stream-
    name-prefix server/
$ jq -r .message /var/tmp/manual-standin/cloudwatch/server2/production/server/b09bf0476120469f9b0cd4
    349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef.jsonl | jq -c '{level,event,component,stage,status}
    | with_entries(select(.value != null))'
{"level":"warn","event":"operation.finished","component":"gateway","stage":"provider_http"}
{"level":"warn","event":"operation.finished","component":"http","stage":"request","status":501}
```

The readiness line stays in the journal only. S3 (level info) holds the info and warn records; CloudWatch (level warn) holds only the two warn records of the HTTP 501 request.

Expected result: Each destination holds exactly the server records at or above its level, as the original lines.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

6. Prepare the AWS templates (LP-5)

Render the role policy, bucket lifecycle and log group templates for the actual bucket, prefix, group, region and account with server2/deploy/logging/aws_templates.py. Then apply them with administrator credentials using the commands in the guide (section Prepare AWS for the destinations); put-bucket-lifecycle-configuration replaces the whole configuration, so read it first. The capture shows synthetic names; nothing was applied to AWS.

```
$ python3 -I server2/deploy/logging/aws_templates.py --bucket tw-server2-123456789012-apne2 --prefix
     raw/v2/logs --group /server2/production --region ap-northeast-2 --account 123456789012 --out
    $OUT
aws_templates: SERVER2_LOG_S3_PREFIX: must not start inside a weather data prefix: raw, index,
    packs, summary, state, budgets, geocode

$ python3 -I server2/deploy/logging/aws_templates.py --bucket tw-server2-123456789012-apne2 --prefix
     logs/server2 --group /server2/production --region ap-northeast-2 --account 123456789012 --out
    $OUT
aws_templates: rendered 3 files into $OUT

$ jq -c '.Statement[] | {Sid, Action, Resource}' $OUT/role-policy.json
{"Sid":"PutLogObjects","Action":["s3:PutObject"],"Resource":"arn:aws:s3:::tw-
    server2-123456789012-apne2/logs/server2/*"}
{"Sid":"WriteLogStreams","Action":["logs:CreateLogStream","logs:PutLogEvents"],"Resource":"arn:aws:l
    ogs:ap-northeast-2:123456789012:log-group:/server2/production:log-stream:*"}
{"Sid":"FindLogStreams","Action":["logs:DescribeLogStreams"],"Resource":["arn:aws:logs:ap-
    northeast-2:123456789012:log-group:/server2/production","arn:aws:logs:ap-
    northeast-2:123456789012:log-group:/server2/production:*"]}

$ jq -c '.Rules[] | {ID, Filter, Expiration, NoncurrentVersionExpiration,
    AbortIncompleteMultipartUpload} | with_entries(select(.value != null))' $OUT/bucket-
    lifecycle.json
{"ID":"server2-logs-expire","Filter":{"Prefix":"logs/server2/"},"Expiration":{"Days":90},"Noncurrent
    VersionExpiration":{"NoncurrentDays":1},"AbortIncompleteMultipartUpload":{"DaysAfterInitiation":
    1}}
{"ID":"server2-logs-delete-
    markers","Filter":{"Prefix":"logs/server2/"},"Expiration":{"ExpiredObjectDeleteMarker":true}}

$ jq -c . $OUT/log-group.json
{"logGroupName":"/server2/production","logGroupClass":"STANDARD","retentionInDays":30}
```

A prefix inside a weather data prefix is refused and nothing is written. The rendered policy grants only PutObject under the log prefix, stream creation, event writing and stream lookup in the one group; the lifecycle covers only the log prefix; the group keeps 30 days.

Expected result: Three JSON files without placeholders, ready for the owner to apply and read back.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

7. Watch a destination fail and recover (LP-2)

Read the collector's counters on the host (curl -s http://127.0.0.1:9598/metrics) and its status record, written every 60 s to the server2 journal and shipped to every destination. A destination whose oldest unconfirmed record is older than twice its delivery window (S3 120 s, CloudWatch 20 s) is shown as behind. The capture makes S3 fail with the stand-in's fault switch for over two minutes while CloudWatch keeps working, then lets it recover.

```
# S3 has been failing for over two minutes (stand-in fault); CloudWatch works
$ curl -s http://127.0.0.1:9598/metrics | grep -E
    '^vector_component_(sent_events|errors)_total.*component_id="(s3|cloudwatch)"' | sed -E
    's/,[a-z_]+="[^"]*"//g; s/ [0-9]{13}$//'
vector_component_sent_events_total{component_id="cloudwatch"} 9

$ systemctl start server2-logs-status.service; journalctl --namespace=server2 -u server2-logs-
    status.service -o cat | tail -1 | jq -c '{level, s3: .destinations.s3, cloudwatch:
    .destinations.cloudwatch}'
{"level":"warn","s3":{"behind":true,"dropped_events":0,"failed_requests":0,"journal_overruns":0,"sen
    t_events":0,"unshipped_seconds":134},"cloudwatch":{"behind":false,"dropped_events":0,"failed_req
    uests":0,"journal_overruns":0,"sent_events":9,"unshipped_seconds":0}}

# S3 recovers; the records written meanwhile are delivered from the journal
$ systemctl start server2-logs-status.service; journalctl --namespace=server2 -u server2-logs-
    status.service -o cat | tail -1 | jq -c '{level, event, destinations}'
{"level":"info","event":"collector.status","destinations":{"cloudwatch":{"behind":false,"dropped_eve
    nts":0,"failed_requests":0,"journal_overruns":0,"sent_events":11,"unshipped_seconds":0},"s3":{"b
    ehind":false,"dropped_events":0,"failed_requests":0,"journal_overruns":0,"sent_events":22,"unshi
    pped_seconds":0}}}

$ ss -ltnH 'sport = :9598' | awk '{print $4}'
127.0.0.1:9598
```

While S3 fails, the collector keeps retrying, so its failure counter stays at zero; the status record is warn with s3 behind (134 s unconfirmed) and CloudWatch current. After recovery the records written meanwhile arrive from the journal and the status is info again. The counters listen on 127.0.0.1 only.

Expected result: An outage is visible in the status record while it lasts, and recovery needs no action.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

8. Start a new instance from the boot configuration (LP-3)

Generate the user data with boot.py for a commit that is on GitHub (normally the merged master commit) and the destination settings, and give user-data.sh.gz to the launch template or instance. At the first boot it downloads the pipeline files from that commit, refuses any file whose SHA-256 differs, installs the pinned collector and starts it; the release procedure then installs server2 and its .env. Nobody logs in, and the instance holds no AWS key.

```
# On the operator's machine: generate the user data for a commit on GitHub
$ python3 -I server2/deploy/logging/boot.py --revision fd439c85e59f4f0182ec2508645526b02f94f831 \
    --sinks local,s3,cloudwatch --prefix logs/server2 --group /server2/check \
    --bucket server2-logs-check --region ap-northeast-2 --out user-data
boot.py: revision fd439c85e59f4f0182ec2508645526b02f94f831; user-data.sh 2436 bytes, user-data.sh.gz
     1322 bytes (EC2 limit 16384)
$ grep -E '^(base|fetch (render|install))' user-data/user-data.sh
base=https://raw.githubusercontent.com/WizardFactory/TodayWeather/fd439c85e59f4f0182ec2508645526b02f
    94f831/server2/deploy/logging
fetch install.sh fd68b5271315f8447015add5cb8eb2b7ee10ca57f78cd94553730329c16699ee
fetch render.py b704ecc985402e9977e94d16232d669cd84aad9ff5d16b9d7008cbefc5702462
# On the new instance after its first boot (no login was needed for any of this)
$ systemctl is-enabled server2.service server2-logs.service
enabled
enabled
$ systemctl is-active server2-logs.service
active
$ cat /etc/systemd/system/server2-logs.service.d/destinations.conf
[Service]
# Destination settings from the boot configuration; they take precedence over .env.
Environment="SERVER2_LOG_SINKS=local,s3,cloudwatch"
Environment="SERVER2_LOG_S3_LEVEL=info"
Environment="SERVER2_LOG_CLOUDWATCH_LEVEL=warn"
Environment="SERVER2_LOG_S3_PREFIX=logs/server2"
Environment="SERVER2_LOG_CLOUDWATCH_GROUP=/server2/check"
Environment="SERVER2_S3_BUCKET=server2-logs-check"
Environment="SERVER2_S3_REGION=ap-northeast-2"
$ journalctl --namespace=server2 -u server2-logs.service -o cat | grep collector.started | tail -n 1
{"boot_id": "0e808132cf6540638331239 87c8d9b06", "collector_schema": 1, "collector_version": "vector
    0.61.0", "event": "collector.started", "instance_id": "i-0c0ffee0000000001",
    "instance_life_cycle": "on-demand", "level": "info", "sinks": ["local", "s3", "cloudwatch"],
    "timestamp": "2026-11-14T14:00:22.984091Z"}
```

The user data names the commit and each file's hash (about 1.3 KB of EC2's 16 KB). On the new instance both units are enabled, the collector runs, the destination settings are a unit drop-in, and the start record names the EC2 instance ID, its life cycle and the boot.

Expected result: The collector ships from its first record; every instance of the launch template uses the same user data and writes its own streams and keys.

server2 host log pipeline: install, select destinations, read logs, prepare AWS, watch for loss, start new instances unattended, survive a Spot interruption

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, renders the AWS templates (LP-5) that the owner applies, watches one destination fail and recover (LP-2), starts a new instance from the boot configuration alone (LP-3), and sees a Spot interruption stop the server and then the collector (LP-4). Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names; for step 8 a second, new VM had nothing installed; for steps 8 and 9, instance metadata (instance ID, life cycle, instance-role credentials) was the stand-in server2/deploy/logging/imds_standin.py. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

9. A Spot interruption stops the server, then the collector (LP-4)

Nothing to do: on a Spot instance server2-logs-watch.service polls instance metadata every 4 s. When EC2 announces an interruption, it writes the interruption record, stops server2.service, then server2-logs.service, whose drain waits at most 10 s before the collector's own flush. Afterwards, read the watcher's lines and the collector's drained record in the server2 journal namespace.

```
# A Spot instance: the watcher polls instance metadata every 5 s
$ journalctl --namespace=server2 -u server2-logs-watch.service -o cat
server2 interruption watcher: watching for a Spot interruption notice every 5 s
# EC2 announces an interruption (here: the metadata stand-in)
# ... 13 s later
$ journalctl --namespace=server2 -u server2-logs-watch.service -o cat | grep -v watching
{"action": "terminate", "action_time": "2026-11-14T17:43:49Z", "collector_schema": 1, "event":
    "collector.interruption_notice", "level": "warn", "message": "Spot interruption notice: stopping
     server2, then the log collector", "timestamp": "2026-11-14T17:41:51.981382Z"}
server2 interruption stop: server2.service stopped in 0.0 s (exit 0), server2-logs.service by 10.3 s
$ systemctl is-active server2.service server2-logs.service
inactive
inactive
$ journalctl --namespace=server2 -u server2-logs.service -o cat | grep collector.drained | tail -n 1
{"collector_schema": 1, "destinations": {"cloudwatch": {"drained": true, "waited_seconds": 10.0},
    "s3": {"drained": false, "waited_seconds": 10.0}}, "event": "collector.drained", "level":
    "warn", "timestamp": "2026-11-14T17:42:02.170308Z"}
$ journalctl -u server2.service -u server2-logs.service -o short-precise --no-pager | grep -E
    'Stopped server2.service|Stopping server2-logs|Stopped server2-logs' | tail -n 3 | cut -c8-110
02:41:52.021981 lima-lp2977 systemd[1]: Stopped server2.service - server2 weather origin.
02:41:52.037997 lima-lp2977 systemd[1]: Stopping server2-logs.service - server2 log collector...
02:42:02.271506 lima-lp2977 systemd[1]: Stopped server2-logs.service - server2 log collector.
```

The interruption record names the announced action and time. The server stopped first, the collector about 10 s later. CloudWatch had confirmed everything; S3's open object was sent by the collector's flush (drained false, so the record is warn). The whole sequence took 13 s of the two-minute notice; the budget is 86 s.

Expected result: Every record up to the server's stop reaches each destination, and the interruption record comes before the server's shutdown records. On an On-Demand instance the watcher says it is idle and stops nothing.